

CSA0306 • DATA STRUCTURES

Dynamic Waiting List Management for Railway Reservation Systems

A Linked-List Based Approach — Complexity Analysis & Justification

CO1 – Assessment Tool 3 – Technical Presentation



Presented By: Variganji Sumanth Babu

Reg No: 192524447

SIMATS Engineering

PROBLEM STATEMENT

The Railway Waiting-List Challenge

Scenario

When train tickets are fully booked, new passengers are placed on a Waiting List (WL) in the order they book. As confirmed passengers cancel, every waiting passenger below them must move up by exactly one position — instantly and correctly.

Why Dynamic Updates Are Required

- Cancellations happen at random positions, at any time, in real time.
- Every passenger below the cancelled position must shift up by one rank.
- The list size is unpredictable — it grows and shrinks continuously.
- Position lookups and updates must stay fast even as WL length varies.
- Wrong or delayed updates cause incorrect boarding/refund decisions.



Passengers form a waiting queue at the reservation counter

Position Shift Example

WL1 – Arjun

WL2 – Priya

WL3 – Kabir

WL4 – Sailaja



After Cancellation

WL1 – Arjun

WL2 – Kabir

WL3 – Sailaja

Every passenger below WL2 shifts up by one position instantly.

What This Presentation Aims To Do



01

Identify the Right Structure

Select a data structure that models a real-time, size-varying railway waiting list.



02

Explain the Working Mechanism

Describe how insertion, cancellation and position updates operate on the structure.



03

Present a Working Algorithm

Provide clean pseudocode with a worked cancellation example.



04

Perform Complexity Analysis

Derive time and space complexity for insertion, deletion, search and update.



05

Justify the Choice

Compare against Array and Queue to prove the structure is the best fit.

Singly Linked List



Why a Linked List?

- Size is unknown in advance and changes constantly as passengers book and cancel.
- Deletion at any position needs only pointer updates — no bulk shifting of elements.
- Nodes are allocated dynamically, so memory is used only as needed.
- Sequential traversal naturally mirrors WL1, WL2, WL3 ... ranking.
- New passengers are appended in $O(1)$ using a maintained tail pointer.

Linked List Structure of the Waiting List



Each node stores passenger data and a pointer to the next node. Cancellation removes a node by re-linking its neighbours in $O(1)$ once located.

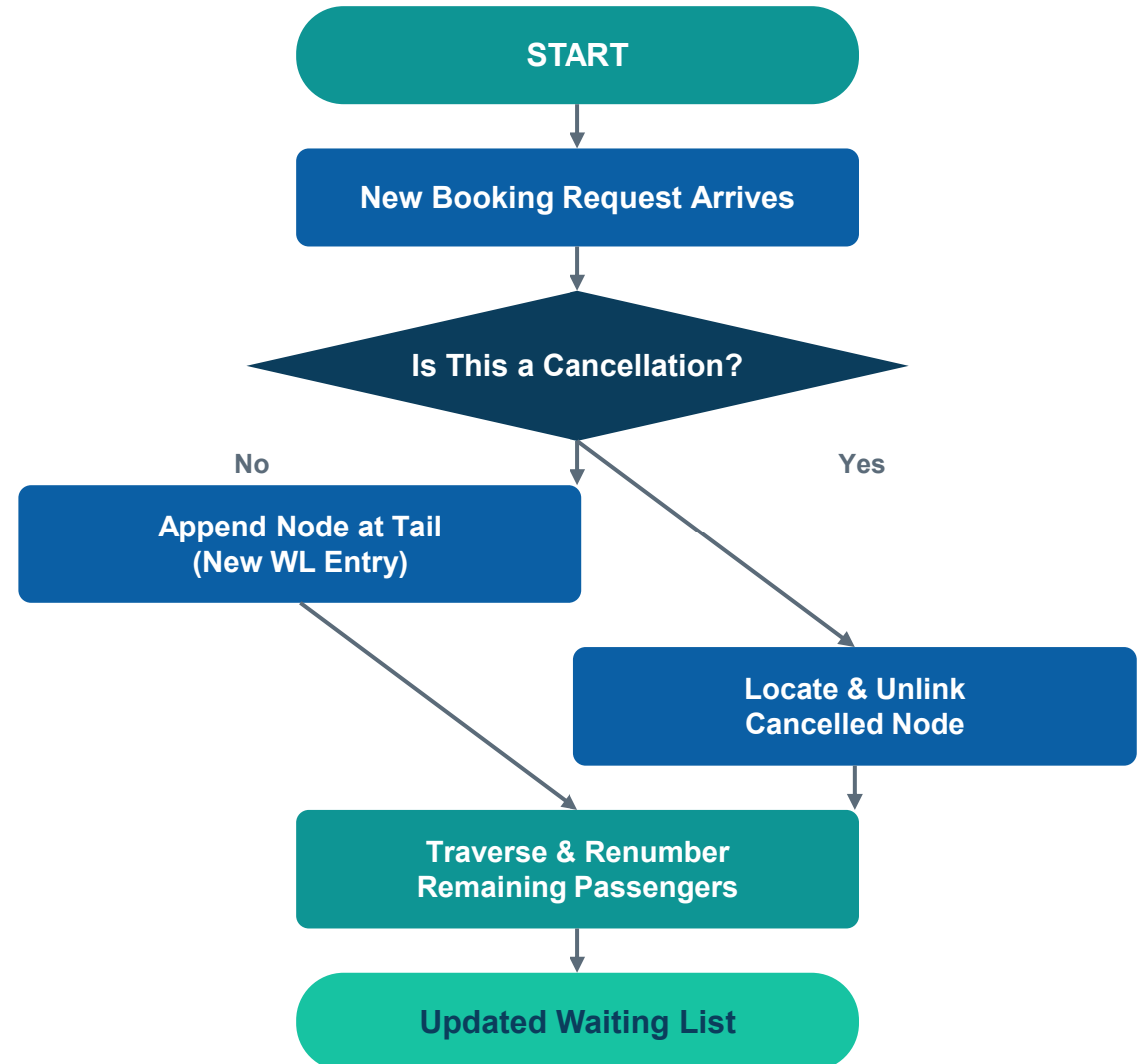
```
struct Node {  
    int passengerID;  
    string name;  
    int position;  
    Node* next;  
};
```

WORKING PRINCIPLE

Insertion, Cancellation & List Update

- New booking: a node is created and appended at the tail — becomes the last WL rank.
- Cancellation: the system traverses to the node just before the target position.
- Re-linking: the previous node's pointer is redirected to skip the cancelled node.
- Renumbering: the list is traversed once to reassign WL1, WL2, WL3 ... in order.
- Confirmation: if a berth frees up, the head node is promoted to a confirmed seat.

All updates are performed in a single traversal pass, keeping the operation efficient even for a long waiting list.



ALGORITHM

Pseudocode & Worked Example

Pseudocode — Cancellation & Update

PROCEDURE CancelBooking(pos) :

```
prev ← NULL
curr ← HEAD
count ← 1
```

```
WHILE curr ≠ NULL AND count ≠ pos:
    prev ← curr
    curr ← curr.next
    count ← count + 1
```

```
IF curr ≠ NULL:
    IF prev = NULL:
        HEAD ← curr.next          // cancel WL1
    ELSE:
        prev.next ← curr.next      // unlink node
    FREE(curr)
```

RenumberList()

PROCEDURE RenumberList() :

```
node ← HEAD; rank ← 1
WHILE node ≠ NULL:
    node.position ← rank
    rank ← rank + 1; node ← node.next
```

Worked Example

Passenger at WL2 cancels their ticket.

Before Cancellation:



After Unlink + Renumber:



Step 1 — Traverse to node before WL2, unlink Priya's node.

Step 2 — Kabir's node becomes the new WL2 link target.

Step 3 — Single traversal rennumbers Kabir → WL2, Sailaja → WL3.

Time and Space Complexity

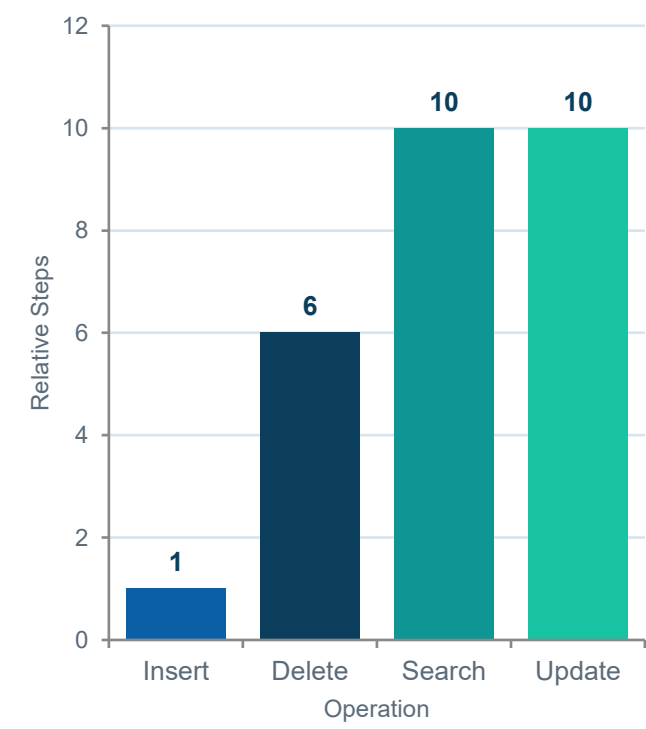
Operation	Time Complexity	Space Complexity	Notes
Insertion (append new booking)	$O(1)^*$	$O(1)$	Constant with a maintained tail pointer
Deletion (cancellation)	$O(n)$	$O(1)$	Locating the node is $O(n)$; the unlink itself is $O(1)$
Search (find passenger / position)	$O(n)$	$O(1)$	Linear scan from HEAD in the worst case
Update (renumber after change)	$O(n)$	$O(1)$	One full traversal reassigns all ranks

**Insertion is $O(1)$ only when a tail pointer is maintained; without it, appending requires an $O(n)$ traversal to the last node.*

Space Overview: Total space is $O(n)$ for n waiting passengers — one node per passenger, each carrying one extra pointer field. No extra memory is pre-allocated or wasted, unlike a fixed-size array.

Relative Operation Cost

Steps required for $n = 10$ passengers



Linked List vs Array vs Queue

Criteria	Array	Linked List	Queue (Array-based)
Insertion (at rank)	$O(n)$ — shift elements	$O(1)$ at tail / $O(n)$ mid-list	$O(1)$ only at rear
Deletion (mid-list)	$O(n)$ — shift elements	$O(n)$ traverse + $O(1)$ unlink	$O(n)$ — not native to queue
Memory Usage	Fixed / pre-allocated, may waste space	Dynamic, exact-fit + pointer overhead	Fixed / pre-allocated
Flexibility for WL	Poor — resizing is costly	Excellent — grows/shrinks freely	Poor — no random-position cancel



Array

Rigid size and costly shifts make it unsuitable for a fluctuating waiting list.



Linked List

Dynamic sizing and $O(1)$ unlinking make it the best fit for this problem.



Queue

Only supports front/rear operations — cannot cancel from the middle efficiently.

CRITICAL EVALUATION

Advantages and Limitations



Advantages

- Dynamic size — grows and shrinks with real bookings, no wasted memory.
- Efficient mid-list deletion — cancellation needs only pointer re-linking.
- No large-scale shifting of elements, unlike arrays.
- Natural fit for sequential rank-based ordering (WL1, WL2, WL3...).
- Easily extendable to a doubly linked list for backward traversal.



Limitations

- No random access — reaching the k-th passenger needs $O(n)$ traversal.
- Extra memory per node for storing the pointer field.
- Search and update remain $O(n)$ in the worst case.
- Cache performance is weaker than arrays due to non-contiguous memory.
- Slightly more complex to implement and debug than a plain array.

Where This Pattern Is Used



Railway Reservation Systems

Managing WL/RAC passenger positions as seats are confirmed or cancelled.



Hospital Waiting Lists

Ordering patients for beds or procedures as availability changes.



Customer Service Queues

Routing callers to agents in arrival order, updating live on hang-ups.



Job Scheduling

Operating systems maintaining a dynamic list of processes awaiting CPU time.

CONCLUSION

The Linked List Is the Right Fit

- The railway waiting list is inherently dynamic — its length and content change unpredictably with every booking and cancellation.
- A singly linked list accommodates this directly: $O(1)$ tail insertion, and $O(1)$ unlinking once a cancelled node is located.
- Position renumbering after any change is handled in a single, efficient traversal — $O(n)$ but with minimal, predictable cost.
- Compared to arrays and queues, it alone offers the flexibility to grow, shrink and reorder without wasted memory or bulk shifting.
- Overall, the linked list delivers the best balance of efficiency, memory economy and real-world correctness for this problem.

Hence, a Singly Linked List is the most suitable data structure for managing a dynamically changing railway reservation waiting list.

At a Glance

 $O(1)$

Insertion

 $O(n)$

Deletion

 $O(n)$

Search

 $O(n)$

Total Space

Academic Integrity Declaration



I, Variganji Sumanth Babu (Reg No: 192524447), certify that this presentation is my original work and has been prepared according to the assessment guidelines provided by SIMATS.

Student Signature

V. Sumanth Babu

Date

15th July 2026

Variganji Sumanth Babu | Reg No: 192524447 | CSA0306 – Data Structures